

Arquitectura de las Computadoras I

Examen en máquina

Ing. Walter Lozano

Ing. Alejandro Rodríguez Costello

Introducción:

La **rotación de un vector** es una operación que mueve elementos circularmente. En una rotación a la izquierda de k posiciones, los primeros k elementos se mueven al final y el resto se desplaza k posiciones a la izquierda.

Formalmente, si $v = [v_0, v_1, \dots, v_{n-1}]$, una rotación izquierda de k posiciones produce:

$$v' = [v_k, v_{k+1}, \dots, v_{n-1}, v_0, v_1, \dots, v_{k-1}]$$

Por ejemplo, el vector $v = [1, 2, 3, 4, 5, 6]$ rotado 2 posiciones a la izquierda resulta en:

$$v' = [3, 4, 5, 6, 1, 2]$$

La rotación a la derecha funciona de manera inversa. Esta operación es útil en procesamiento de señales, criptografía y manipulación de buffers circulares.

Un algoritmo eficiente usa **tres inversiones**: invertir $[0\dots k-1]$, invertir $[k\dots n-1]$, invertir todo $[0\dots n-1]$.

Consigna:

Hacer un programa en assembly de MIPS R2000, respetando las convenciones, que rote un vector k posiciones. Para ello deberá realizar 4 funciones: **rotateVec**, **reverseSegment**, **swap** y **outVec**.

La función **rotateVec** es no leaf e implementa la rotación usando el algoritmo de tres inversiones. Se llama desde main y recibe puntero al vector, dimensión, y cantidad de posiciones a rotar (k). Debe llamar a **reverseSegment** tres veces: $[0, k-1]$, $[k, n-1]$ y $[0, n-1]$.

La función **reverseSegment** es no leaf e invierte un segmento del vector. Recibe puntero al vector, índice inicial e índice final del segmento. Debe llamar a **swap** repetidamente para intercambiar elementos simétricos.

La función **swap** es leaf e intercambia dos elementos del vector. Recibe puntero al vector y dos índices de los elementos a intercambiar.

La función **outVec** es leaf y muestra el vector. Recibe puntero al vector y su dimensión.

El programa debe solicitar el vector, la cantidad de posiciones a rotar, mostrar el vector original y el resultado de la rotación. Debe manejar valores de k mayores que n ($k \bmod n$).

Ejemplo de ejecución:

Vector original (dimensión 7):

10 20 30 40 50 60 70

Ingrese cantidad de posiciones a rotar a la izquierda: 3

Después de inversión [0,2]: 30 20 10 40 50 60 70

Después de inversión [3,6]: 30 20 10 70 60 50 40

Después de inversión [0,6]: 40 50 60 70 10 20 30

Vector rotado:

40 50 60 70 10 20 30